A Secure Certificateless Signature Scheme for Cloud-Assisted Industrial IoT

Kyung-Ah Shim, *Member, IEEE*

***Abstract*—Although the cloud-assisted Industrial Internet of Things (IIoT) is emerging as one of the promising technologies to improve personal lifestyles, security and privacy issues are still challenging. Secure certificateless signature (CLS) schemes without certificates for the authentication of public keys are considered suitable for the resource-constrained IIoT security and authentication. Recently, Wang et al. pointed out several security weaknesses on the existing CLS schemes and then constructed a novel efficient CLS scheme based on the blockchain technique and the smart contract. In this work, we show that Wang et al.'s scheme is broken by key recovery attacks: anyone can recover the entire private keys of any users. We then propose a new secure CLS scheme and prove its unforgeability against type I and type II adversaries under the hardness assumption of the Elliptic Curve Diffie-Hellman problem.**

***Index Terms*—Blockchain, certificateless signature (CLS), industrial Internet of Things (IIoT), key recovery attack, signature forgery attack.**

Nomenclature

$params$	System parameters.
H, H_0, H_1	Three collision-resistant hash functions.
(s, P_{pub})	System master secret, a master public key.
ID_i	Identity of a user $\mathcal{U}_i$.
(PK_i, SK_i)	Public key, a private key of $\mathcal{U}_i$, respectively.
S_i	Partial privet key $\mathcal{U}_i$.
$\sigma_i = (U_i, V_i)$	Signature of $\mathcal{U}_i$ on a message m_i.
T_i	Current time stamp.

I. Introduction

A. Internet of Things Security

THE Internet of Things (IoT) enables interactions between things-to-things with various smart devices, such as sensors, radio frequency identification, and actuators to connect, collect, and exchange data over the Internet. In particular, the cloud-assisted industrial Internet of Things (IIoT), a combination of the IIoT and cloud computing, is one of the promising solutions for improving industrial systems. Participants in the cloud-assisted IIoT upload various data collected by IoT devices to cloud servers and enjoy various services by industrial organizations. The cloud-assisted IIoT still suffers from security and privacy issues, such as integrity of transmitted data, authenticity of involved parties, and efficiency of the systems. It is essential to ensure the authenticity and integrity of IIoT data in an efficient manner before processing and analyzing the collected data. The security and privacy issues should be resolved before the actual deployment of the IIoT systems.

B. Limitations and Requirements

Based on the features of IoT devices and networks, we consider the following limitations and requirements.

1) *Limited transmission bandwidth:* Since IoT applications require limited transmission bandwidth, the minimum sizes of signatures and public keys are needed.
2) *Minimal energy consumption:* Due to the prevalence and increased use of battery-operated devices, the evaluation of energy consumptions is important. A long battery lifetime is important for economic vitality. Energy consumption is proportional to the execution time, since $E = Po \times T$, where Po is the power consumption and T is the algorithm's execution time.
3) *Memory constraint:* Most embedded processors are memory constrained, so the smaller the memory footprint, the better.
4) *Performance constraint:* Information in an IoT application must be delivered to the nodes within a time limit so that the nodes can make decisions and take action accordingly. The performance of cryptographic operations on IoT devices is one of the major causes of time consuming. Thus, the faster without requiring additional resources the speed, the better.
5) *Tradeoff:* Depending on these limitations, a tradeoff between performance (cost, resource, latency, etc.) and the required security may be necessary to achieve several functions.

C. Suitability of Certificateless Cryptography for the IoT

There exist three types of cryptography: public-key cryptography, identity (ID)-based cryptography, and certificateless

Manuscript received 15 September 2022; revised 18 September 2023; accepted 29 November 2023. Date of publication 25 January 2024; date of current version 4 April 2024. This work was supported by the National Institute for Mathematical Sciences funded by the Ministry of Science and ICT of Korea under Grant B23710000. Paper no. TII-22-3910.

The author is with the Division of Basic Researches for Industrial Mathematics, National Institute for Mathematical Sciences, Daejeon 34047, South Korea (e-mail: kashim@nims.re.kr).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2023.3343437.

Digital Object Identifier 10.1109/TII.2023.3343437

1551-3203 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

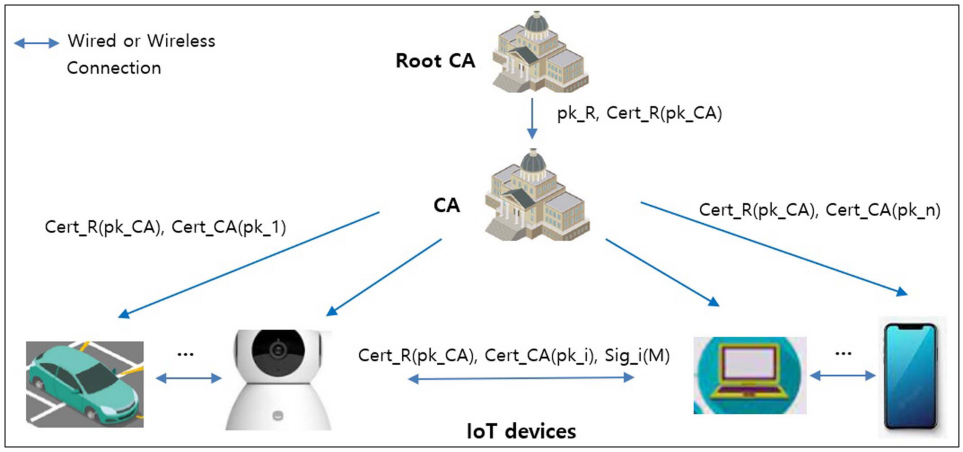


Fig. 1. System model of the PKS schemes in the PKI.

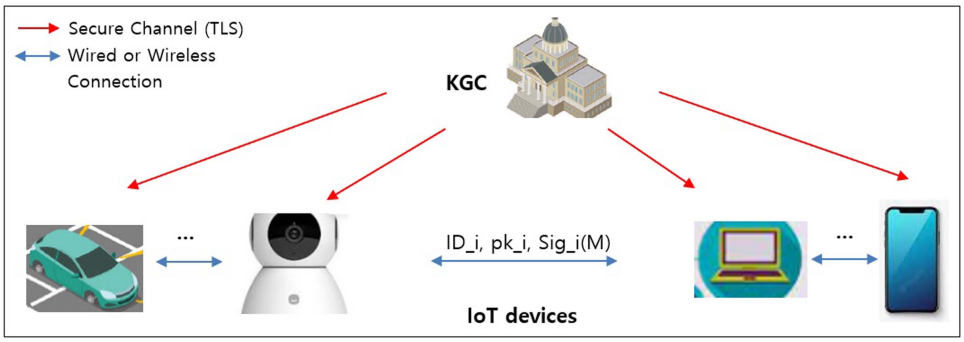


Fig. 2. System model of the CLS schemes.

public-key cryptography. Public-key cryptographic algorithms provide digital signatures and establish secure communication without requiring key predistribution unlike symmetric-key cryptography. In public-key cryptography [1], users' pubic keys are random strings, which need authentication by a certificate authority (CA). Such authentication is realized by using public-key certificates issued by CAs in public-key infrastructure (PKI), which is a set of roles, policies, and procedures needed to create, manage, distribute, use, store, and revoke public-key certificates. It causes the certificate management problems. To remove the problems, Shamir [2] introduced ID-based cryptography, in which a key generation center (KGC) computes users' private keys from the users' unique ID information, such as e-mail address by using KGC's master secret key. However, it causes the key escrow problem. A new concept of certificateless cryptography was proposed by Al-Riyami and Paterson [3] to resolve the problems in two types of cryptography. Certificateless cryptography generates a user's private key by combining the user-chosen secret value with secret information from KGC's master secret key, which solves the two problems. Certificateless public-key cryptography requires no certificate for the authentication of public keys and ciphertext/signatures are transmitted with users' public keys.

Currently, all the IoT systems use the public-key signature (PKS) schemes to ensure authentication, integrity, and nonrepudiation. However, the PKS schemes need the authentication of the public keys realized by the PKI. The problems related to certificate management in the PKI, including revocation, storage, distribution, and the computational cost of certificate verification, are challenging tasks in IoT environments that consist of billions of resource-constrained devices. The basic system models of the PKS and certificateless signature (CLS) schemes are depicted in Figs. 1 and 2, respectively. As seen in Fig. 1, two public-key certificates for the authentication of two public keys of the CA and a node are transmitted to send a signature on a message when the PKS scheme is used. There are two types of certificates for the CA and a node: one is CA's certificate issued by the root CA and the other is a node's certificate issued by the CA. The node's certificate consists of the node's ID, a node's public key, a CA's signature on the public key, and information on expiration date. As seen in Fig. 2, the CLS scheme does not require the public-key certificates, and a node's ID, a node's public key, and a certificateless signature on a message are enough. In terms of communication cost, in the PKS schemes, three signatures and three public keys should be transmitted per message being signed, but two

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

signatures and a public key are enough to send a signature on a message in the CLS schemes. In terms of computational cost, the PKS schemes require three signature verifications, but the CLS schemes require only one signature verification. In order to minimize transmission bandwidth, energy consumption, and performance in IoT systems, the CLS schemes are superior to the PKS schemes. Consequently, the CLS schemes are suitable for IoT systems.

D. Related Works

There have been proposed a number of CLS schemes for IoT, IIoT, VANETs, health-care wireless medical sensor networks, and resource-constrained devices [4], [5], [6], [7], [8], [9], [10], [11], [12]. Some of them are based on the bilinear pairings, some do not use the bilinear pairings, some are proven in random oracle models, and others are proven in standard models. There has also been proposed a generic coversion from given standard signature schemes and ID-based signature schemes into CLS schemes [13]. However, most of them were broken or cryptanalyzed: most of them were insecure against type I attacks, type II attacks, or malicious-but-passive attacks [10], [12], [14], [15], [16], [17], [18], [19], [20], [21]. Although the certificates (CLS) schemes remove the need of the certificate for the authentication of public keys by the CA, they still require the centralized KGC to generate the partial private keys of users. Decentralized blockchain with its smart contract can solve the problem for the centralized KGC [22], [23]. Recently, Wang et al. [12] pointed out several security weaknesses on most of the existing CLS schemes [4], [6], [11], [15], [21] and then constructed a novel efficient CLS scheme based on the blockchain technique and the smart contract. Their scheme supported a fully decentralized and distributed denial-of-service mitigated KGC by transforming the logic of the KGC into a smart contract code. In this work, we show that Wang et al.'s scheme is broken by key recovery attacks: anyone can recover the entire private keys of any users.

E. Contributions

The main contributions of this article are summarized as follows.

1) *Key recovery attacks on Wang et al.'s scheme:* We show that Wang et al.'s scheme is broken by key recovery attacks: anyone can recover the partial private key and the private key of any users from only two signatures. Thus, anyone can generate valid signatures of any messages under any users from the whole private keys. In [12], they proved unforgeability of the scheme against type I and type II adversaries under the hardness assumption of the Elliptic Curve Discrete Logarithm problem (ECDLP). Our results mean that the ECDLP is easily solved, so their security proofs are flawed. Consequently, Wang et al.'s scheme is vulnerable to the signature forgery attacks. In order for the blockchain structure to be secure, the security of the adopted signature scheme must be guaranteed first.
2) *Causes and countermeasures:* We discuss causes and countermeasures against our attacks. The cause of our key recovery attacks is that the signature generation of Wang et al.'s scheme requires only secret information without using random values for each signing. This means that the signatures consist of the combination of the fixed private keys, (ppk_i, x_i), and the related hash values, where the private key can be computed consecutively by appropriate simple operations in the key recovery attacks. To prevent our attacks, the CLS scheme should be constructed as a probabilistic algorithm by using random values for each signing.
3) *Construction of a new CLS scheme:* Finally, we construct a new secure CLS scheme for the IIoT and prove unforgeability of our scheme against type I and type II adversaries under the hardness assumption of the Elliptic Curve Diffie-Hellman problem (ECDHP). Unforgeability of most CLS schemes has been proven using the forking lemma [24], which results in very loose security reductions. Our security proofs against the two types of adversaries do not use the forking lemma. Our CLS scheme is designed to be proven without using the forking lemma using three hash functions as random oracles. It allows us to perform appropriate simulations without the use of the forking lemma. As a result, our scheme guarantees stronger security than the proven CLS scheems using the forking lemma.

F. Organization

The rest of this article is organized as follows. We describe Wang et al.'s blockchain-based CLS scheme and show that their scheme is entirely broken in Section II. We construct a new CLS scheme and prove its security in the random oracle model in Section III. In Section IV, we give comparison and performance evaluation. Finally, Section V concludes this article.

II. Security Analysis of Wang et al.'s Scheme

We review Wang et al.'s blockchain-based CLS scheme for IIoT devices [12].

A. System Model

Wang et al.'s blockchain-based CLS scheme consists of the following three entities.

1) *SC-KGC:* A smart contract computes and distributes partial private keys of participated users. Participated users can use the public system parameters.
2) $\mathcal{U}_i$*:* SC-KGC computes partial private keys from the participated users' ID information. After receiving related secret information, a user $\mathcal{U}_i$ generates a valid signature σ_i on a message m_i.
3) $\mathcal{V}_j$*:* A verifier $\mathcal{V}_j$ checks the validity of a signature σ_i on m_i.

The system model of Wang et al.'s blockchain-based CLS scheme is depicted in Fig. 3.

B. Hardness Assumption

We describe two hardness assumptions for security proofs.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

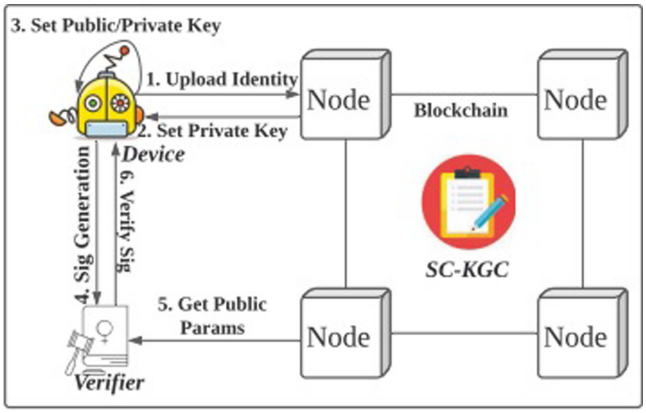


Fig. 3. System model for Wang et al.'s blockchain-based CLS protocol [12].

1) *Elliptic Curve Discrete Logarithm problem:* Given a group G of prime order q, a generator P of G, and $Q = aP$, find a discrete logarithm $a \in_R Z_q^*$. If G is an elliptic curve group, then this problem is ECDLP.
2) *Elliptic Curve Diffie–Hellman problem:* Given a group G, a generator P of G, and (aP, bP) for $a, b \in_R Z_q^*$, compute abP. If G is an elliptic curve group, then this problem is Elliptic Curve Diffie-Hellman problem (ECDHP).

C. Description of Wang et al.'s Blockchain-Based CLS Scheme

1) *Setup:* To initialize the smart contract SC-KGC, an administrator Admin does the following.
   a) For a security parameter $\lambda \in Z^+$, choose a group G of prime order q and its generator P.
   b) Choose $s \in Z_q^*$ randomly as a system master key and calculate $P_{\text{pub}} = sP$ as a master public key.
   c) Select two collision-resistant hash functions $h_1, h_2 : \{0,1\}^* \to Z_q^*$.
   d) Publish system parameters $params = \langle G, P, P_pub, q, h_1, h_2 \rangle$ and a partial private key generation function $Set_Partial_Private_Key$.
2) *Set partial private key:* For the generation of a partial private key ppk_i of $\mathcal{U}_i$ with $ID_i \in \{0,1\}^*$, calculate

$$P_{\text{ID}_i} = ID_i P,\ C_{\text{ID}_i} = ID_i + ID_i P_{\text{pub}}$$

and invoke $Set_Partial_Private_Key$ with inputs $\{P_{\text{ID}}, C_{\text{ID}_i}\}$. When $\mathcal{U}_i$ initializes the function $Set_Partial_Private_Key$, SC-KGC does the following.
   a) Check whether $CTR_{Addr_{\text{ID}_i}} < n$, where $Addr_{\text{ID}_i}$ is $\mathcal{U}_i$'s address on the Ethereum platform and $CTR_{Addr_{\text{ID}_i}}$ is a counter maintained by SC-KGC for a reset threshold value n [25]. If it is correct, then SC-KGC stores ID_i by calculating

$$C_{\text{ID}_i} - sP_{\text{ID}_i} = ID_i + ID_i P_{\text{pub}} - ID_i sP$$
$$= ID_i + ID_i P_{\text{pub}} - ID_i P_{\text{pub}} = ID_i.$$

   Otherwise, SC-KGC rejects the request.
   b) Pick $r_i \in_R Z_q^*$, and calculate $R_i = r_i P$.
   c) Compute $\alpha_i = h_1(P_{\text{pub}}, ID_i, R_i)$

$$ppk_i = (r_i + s\alpha_i) \bmod q$$

   where ppk_i is the partial private key of $\mathcal{U}_i$ and sends $xppk_i = (ppk_i \oplus ID_i)$ to $\mathcal{U}_i$.
3) *Set public/private key:* To generate a public/private key pair $PK_i/SK_i, \mathcal{U}_i$ does the following.
   a) Compute the partial private key from $xppk_i$

$$ppk_i = ID_i \oplus xppk_i.$$

   b) To confirm the validity of ppk_i, check the equation

$$ppk_i P = a_i P_{\text{pub}} + R_i.$$

   If it holds, ppk_i is valid.
   c) Choose $x_i \in_R Z_q^*$ as a secret value.
   d) Calculate $X_i = x_i P$ and set $PK_i = (X_i, R_i)$ as a public key and $SK_i = (x_i,\ ppk_i)$ as a private key.
4) *Signature generation:* To generate a signature σ_i on a message $m_i, \mathcal{U}_i$ does the following.
   a) Compute $\beta_i = h_2(P_{\text{pub}}, ID_i, PK_i, m_i, CT_1)$ and

$$sig_i = (ppk_i - \beta_i x_i) \bmod q$$

   where CT_1 is the current time stamp. Then, $\sigma_i = (PK_i, sig_i)$ is a signature of m_i.
   b) Send $< m_i, \sigma_i = (PK_i, sig_i), CT_1 >$ to $\mathcal{V}_j$.
5) *Signature verification:* To check the validity of the signature σ_i, $\mathcal{V}_j$ does the following.
   a) From SC-KGC, fetch $params$ and R_i.
   b) Check the correctness of the time stamp CT_1. If it is not valid, then received messages are discarded.
   c) Otherwise, calculate

$$\alpha_i = h_1(P_{\text{pub}}, ID_i, R_i)$$
$$\beta_i = h_2(P_{\text{pub}}, ID_i, PK_i, m_i, CT_1).$$

   d) Verify the validity of the equation

$$sig_i P = R_i + \alpha_i P_{\text{pub}} - \beta_i X_i.$$

   Accept σ_i if it holds; otherwise, reject it.

D. Key Recovery Attacks on Wang et al.'s Scheme

Here, we show that Wang et al.'s scheme is insecure against key recovery attacks: anyone can recover the private keys of any users from only two signatures.

Let $\mathcal{A}$ be an adversary who can be anyone who wants to generate a valid signature on any messages under $\{PK_i, ID_i\}$.

1) Suppose that $\mathcal{A}$ gets two valid signatures sig_i and sig_i' on two different messages m and m' under $\{ID_i, PK_i\}$, respectively, where

$$sig_i = ppk_i - \beta_i x_i,\ sig_i' = ppk_i - \beta_i' x_i.$$

   Then, $\mathcal{A}$ can recover the private key (ppk_i, x_i) as follows.
   a) First, $\mathcal{A}$ computes

$$x_i = (-\beta_i + \beta_i')^{-1}(sig_i - sig_i')$$

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

from the following operations and the known values β_i, β'_i, sig_i, and sig'_i as

$$sig_i - sig'_i = ppk_i - \beta_i x_i - ppk_i + \beta'_i x_i$$
$$= -\beta_i x_i + \beta'_i x_i = (-\beta_i + \beta'_i) x_i.$$

b) Next, $\mathcal{A}$ obtains ppk_i from the following operation and the known values x_i, β_i, and sig_i as

$$sig_i + \beta_i x_i = ppk_i.$$

2) Finally, $\mathcal{A}$ gets the whole private key (ppk_i, x_i). Thus, $\mathcal{A}$ can generate a signature on any messages under $\{ID_i, PK_i\}$ at any time.

E. Causes and Countermeasures

We discuss causes and countermeasures against our attacks. In Wang et al.'s scheme, anyone can recover the private keys of any users from only two signatures. The cause of our attacks is that their scheme uses only the private key without requiring random values for each signing. The partial private key ppk_i is generated by using a random value r_i, but, after determining ppk_i, the user $\mathcal{U}_i$ utilizes the same ppk_i for every signing. Thus, the signature consists of the combination of the fixed private key, (ppk_i, x_i), and its related hash value, which leads to the full private key recovery. Finally, anyone who recovers the private key can forge signatures of any messages at any time.

There are two types of adversaries, type I and type II, in the security models for the CLS schemes. Security against the type I adversary means that a malicious user who knows the user private key cannot forge signatures. Security against the type II adversary means that a malicious KGC who knows the system master secret cannot forge signatures. Therefore, for a secure CLS scheme, even though one of them (KGC and user) is compromised, unforgeability should be guaranteed. However, in Wang et al.'s scheme, anyone without abilities of type I and type II adversaries can compute (ppk_i, x_i); thus, it is trivial that the type I and type II adversaries can obtain the private key in the same way. Consequently, their scheme is insecure against the signature forgery attacks. Despite their proofs for unforgeability against the type I and type II adversaries under the hardness assumption of the ECDLP in [12], our results show that the ECDLP is easily solved, so their security proofs are flawed. For the blockchain-based scheme to be secure, the security of adopted signature scheme must be guaranteed first. To prevent these attacks, the CLS scheme should be constructed as a probabilistic algorithm by using random values for each signing.

III. New Secure CLS Scheme for the IIoT

Now, we construct a new secure CLS scheme to prevent our attacks and then prove its security in the two types of security models.

A. Our Construction: $\mathcal{ECLS}$

Now, we propose a new secure CLS scheme, $\mathcal{ECLS}$, based on the bilinear pairings.

1) $\mathcal{ECLS}$:

1) *Setup:* For a security parameter $\lambda \in \mathbb{Z}^+$, KGC outputs system parameters as follows.
   a) Choose a bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$, where $q = |\mathbb{G}_1| = |\mathbb{G}_2|$ is prime, and two generators P and Q of $\mathbb{G}_1$.
   b) Choose $s \in \mathbb{Z}_q^*$ as a master secret key and calculate a master public key $P_{\text{pub}} = sP$.
   c) Select three secure hash functions $H : \{0,1\}^* \to \mathbb{G}_1$, $H_0 : \mathbb{G}_1 \to \mathbb{G}_1$, and $H_1 : \{0,1\}^* \to \mathbb{G}_1$.
   d) Publish the system parameters
   $$params = \langle p, \mathbb{G}_1, \mathbb{G}_2, P, Q, P_{\text{pub}}, H, H_0, H_1 \rangle$$
   and keep the master secret key s secure.
2) *Partial-Private-Key-Extract:* KGC computes a user's partial private key as follows.
   a) Compute $S_i = sH(ID_i)$ as partial private key.
   b) KGC transmits S_i to the user $\mathcal{U}_i$. Then, $\mathcal{U}_i$ can check its validity as
   $$e(S_i,\ P) = e(Q_i,\ P_{\text{pub}}).$$
3) *Set-Private-Extract:* A user $\mathcal{U}_i$ picks a random value $x_i \in \mathbb{Z}_q^*$ as a private key usk_i.
4) *Public-Key-Gen:* A user $\mathcal{U}_i$ computes $PK_i = x_i P$ as a public key and sets $SK_i = (x_i, S_i)$ as a full private key.
5) *Sign:* A user $\mathcal{U}_i$ computes a signature σ_i on a message $m_i \in \{0,1\}^*$ as: pick $u_i \in \mathbb{Z}_q^*$ and compute
   $$U_i = u_i P,\ V_i = S_i + x_i H_1(m_i, T_i) + u_i H_0(Q)$$
   where T_i is a current time stamp. Then, $\sigma_i = (U_i, V_i)$ is a signature of m_i under $\{ID_i, PK_i, T_i\}$.
6) *Verify:* Given a signature σ_i of m_i for $\{ID_i, PK_i, T_i\}$, a verifier checks the following equality:
   $$e(V_i,\ P) = e(H(ID_i), P_{\text{pub}}) e(PK_i, H_1(m_i, T_i))$$
   $$e(U_i, H_0(Q)).$$

If it holds, accept the signature; otherwise, reject it.

B. Security Proofs

We prove the unforgeability of $\mathcal{ECLS}$ against the type I and type II adversaries under the hardness assumptions of the ECDHP in the random oracle model. We follow the adversarial and security models for the type I and type II adversaries in [12] and [26].

Theorem 1: If the ECDHP is (t', ϵ')-hard, then $\mathcal{ECLS}$ is $(t, q_H, q_{H_0}, q_{H_1}, q_{PK}, q_{PPK}, q_{SK}, q_{RP}, q_S, \varepsilon)$-existentially unforgeable against type I adversaries, for any time t and adversary's advantage ϵ such that

$$\varepsilon \geq \mathrm{e}^2 \cdot (q_H + 1) \cdot \varepsilon'$$
$$t' \geq t + (q_H + q_{H_1} + q_{PK} + q_{PPK} + 4q_S + 1) \cdot T_{\text{SM}}$$

where e is the base of the natural logarithm and T_{SM} is time for performing a scalar multiplication in $\mathbb{G}_1$.

Proof: Assume that an instance of the ECDHP, $(P,\ aP,\ bP)$, is given to a type I forger, $\mathcal{A}_I$. We will build an algorithm $\mathcal{B}$,

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

which returns a computational Diffie–Hellman (CDH) solution, abP, using $\mathcal{A}_I$.

[Setup]: The algorithm $\mathcal{B}$ sets $P_{\text{pub}} = aP$ as a master public key from the instance.

The forger $\mathcal{A}_I$ can request queries to random oracles H, H_0, and H_1, and *Public-Key-Gen*, *Partial-Private-Key-Extract*, *Private-Key-Extract*, *Replace-Public-Key*, and *Sign* oracles. The algorithm $\mathcal{B}$ answers these queries via the following simulation.

[H-Queries]: For an H-query at ID_i, $\mathcal{B}$ maintains a list of tuples $(c_i, ID_i, \beta_i, W_i)$ in H-list as: $\mathcal{B}$ selects a random coin $c_i \in \{0, 1\}$ with $Pr[c_i = 0] = \frac{1}{q_H+1}$.

1) If $c_i = 0$, then $\mathcal{B}$ picks $\beta_i \in_R \mathbb{Z}_q^*$, calculates $W_i = \beta_i(bP)$ from the instance, adds a tuple $(c_i, ID_i, \beta_i, W_i)$ to the H-list, and outputs $H(ID_i) = W_i$.
2) If $c_i = 1$, then $\mathcal{B}$ picks $\beta_i \in_R \mathbb{Z}_q^*$, calculates $W_i = \beta_i P$, adds a tuple $(c_i, ID_i, \beta_i, W_i)$ to the H-list, and outputs $H(ID_i) = W_i$.

[H_0-Queries]: For an H_0-query at Q, $\mathcal{B}$ selects $\alpha \in_R \mathbb{Z}_q^*$, calculates $W = \alpha P$, adds a tuple (Q, α, W) to the H_0-list, and outputs $H_0(Q) = W$.

[Public-Key-Gen Queries]: For this query at ID_i, $\mathcal{B}$ keeps a list of tuples (ID_i, PK_i, usk_i) in PK-list as: $\mathcal{B}$ selects a random value $x_i \in \mathbb{Z}_q^*$ as a user private key usk_i, calculates $PK_i = x_i P$, adds the tuple (ID_i, PK_i, usk_i) to the PK-list, and outputs a user public key PK_i.

[Partial-Private-Key-Extract Queries]: When $\mathcal{A}_I$ asks a partial private key of ID_i, $\mathcal{B}$ keeps a list of tuples (ID_i, S_i) in PPK-list as: $\mathcal{B}$ first finds a tuple $(c_i, ID_i, \beta_i, W_i)$ in the PPK-list and does the following.

1) If $c_i = 1$, then $\mathcal{B}$ calculates $S_i = \beta_i(aP)$, stores (ID_i, S_i) in PPK-list, and outputs S_i.
2) If $c_i = 0$, then $\mathcal{B}$ terminates with failure.

[Private-Key-Extract Queries]: For this query on a user private key of ID_i, $\mathcal{B}$ finds the associated tuple (ID_i, PK_i, usk_i) from the PK-list and returns usk_i a user private key.

[Replace-Public-Key Queries]: When $\mathcal{A}_I$ wants to replace a user public key PK_i with PK_i', $\mathcal{B}$ finds the related to the tuple (ID_i, PK_i, usk_i) from the PK-list and then sets

$$PK_i = PK_i',\ usk_i = \perp$$

and stores $(ID_i, PK_i', \perp)$ in the PK-list.

[H_1-Queries]: For an H_1-query at m_i, $\mathcal{B}$ keeps a list of tuples (m_i, γ_i, W_i') in H_1-list as: $\mathcal{B}$ selects $\gamma_i \in_R \mathbb{Z}_q^*$, computes $W_i' = \gamma_i P$, adds a tuple (m_i, γ_i, W_i') to H_1-list, and outputs $H_1(m_i) = W_i'$.

[Sign-Queries]: When $\mathcal{A}_I$ requests a signature of m_i, $\mathcal{B}$ finds the associated tuples (Q, α, W) from the H_0-list, $(c_i, ID_i, \beta_i, W_i)$ from the H-list, (m_i, γ_i, W_i') from the H_1-list, and (ID_i, PK_i, usk_i) from the PK-list,

1) If $c_i = 1$, then $\mathcal{B}$ picks $u_i \in_R \mathbb{Z}_q^*$ and computes

$$U_i = u_i P,\ V_i = \beta_i P_{\text{pub}} + usk_i W_i' + u_i W.$$

Then, $\mathcal{B}$ outputs $\sigma_i = (U_i, V_i)$.

2) If $c_i = 0$, then $\mathcal{B}$ terminates with failure.

All outputs to the Sign oracle are valid as they pass the verification equation. If $\mathcal{B}$ aborts neither the result of $\mathcal{A}_I$'s Partial-Private-Key-Extract query nor the result of $\mathcal{A}_I$'s Sign query, then $\mathcal{A}_I$ cannot distinguish between the real attack and the simulation.

[Forgery]: Finally, $\mathcal{A}_I$ returns a signature $\sigma^* = (U^*, V^*)$ of messages m^* under $\{ID_i, PK_i\}$.

1) If it is invalid, then $\mathcal{B}$ terminates and reports failure.
2) Otherwise, the query on ID_i already exists on the H-list with a tuple $(c_i, ID_i, \beta_i, W_i)$. In this case, if $c_i = 1$, then $\mathcal{B}$ terminates with failure. If $c_i = 0$, then the associated tuples are $(c_i, ID_i, \beta_i, W_i = \beta_i(bP))$ and (m^*, γ^*, W^*). Then, $\mathcal{B}$ computes

$$abP = \frac{V^* - \gamma^* PK_i - \alpha U^*}{\beta_i}.$$

At last, $\mathcal{B}$ outputs abP as a solution of the CDH instance.

We investigate the probability that $\mathcal{B}$ succeeds.

1) E_1: $\mathcal{B}$ does not abort a result of $\mathcal{A}_I$'s Partial-Private-Key-Extract query.
2) E_2: $\mathcal{B}$ does not abort a result of $\mathcal{A}_I$'s Sign query.
3) E_3: $\mathcal{A}_I$ returns a valid forgery σ^* of m^* under $\{ID_i, PK_i\}$.
4) E_4: Event E_3 occurs, $c_i = 0$ for the tuple containing ID_i in the H-list.

If all of these events happen, then $\mathcal{B}$ succeeds. The probability $Pr[E_1 \wedge E_2 \wedge E_3 \wedge E_4]$ is decomposed as

$$\begin{aligned} Pr[E_1 \wedge E_2 \wedge E_3 \wedge E_4] = Pr[E_1] \cdot Pr[E_2|E_1] \cdot \\ Pr[E_3|E_1 \wedge E_2] \cdot Pr[E_4|E_1 \wedge E_2 \wedge E_3]. \end{aligned} \tag{1}$$

1) Since $\mathcal{A}_I$ can request at most q_H queries to Partial-Private-Key-Extract oracle, the probability that $\mathcal{B}$ does not abort a result of $\mathcal{A}_I$'s Partial-Private-Key-Extract query is at least $(1 - \frac{1}{q_H+1})^{q_H}$; thus

$$Pr[E_1] \geq (1 - \frac{1}{q_H + 1})^{q_H}.$$

2) The probability that $\mathcal{B}$ does not abort a result of $\mathcal{A}_I$'s Sign query is also at least $(1 - \frac{1}{q_H+1})^{q_H}$. Thus

$$Pr[E_2|E_1] \geq (1 - \frac{1}{q_H + 1})^{q_H}.$$

3) If $\mathcal{B}$ aborts neither the result of $\mathcal{A}_I$'s Sign query nor the result of $\mathcal{A}_I$'s Partial-Private-Key-Extract query, then $\mathcal{A}_I$'s view is the same as its view in the real attack. Therefore, we get $Pr[E_3|E_1 \wedge E_2] \geq \varepsilon$.
4) Given that events E_1, E_2, E_3, and E_4 happened, $\mathcal{B}$ will abort if $\mathcal{A}_I$ returns a forgery with $c_i = 1$. Since $\mathcal{A}_I$ could not have requested a Sign query for the output, c_i is independent of $\mathcal{A}_I$'s view. So, we obtain

$$Pr[E_4|E_1 \wedge E_2 \wedge E_3] \geq \frac{1}{q_H + 1}.$$

Finally, $\mathcal{B}$ returns the correct answer with probability at least

$$\begin{aligned} \left(1 - \frac{1}{q_H + 1}\right)^{q_H} \cdot \left(1 - \frac{1}{q_H + 1}\right)^{q_H} \cdot \varepsilon \cdot \frac{1}{q_H + 1} \\ \geq \frac{1}{\mathrm{e}^2} \cdot \frac{\varepsilon}{(q_H + 1)} \geq \varepsilon' \end{aligned}$$

where e is the base of the natural logarithm.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

Running time of $\mathcal{B}$ is equal to running time of $\mathcal{A}_I$ plus time to respond to q_H H-queries, q_{H_1} H_1-queries, q_{PK} Public-Key-Gen queries, q_{PPK} Partial-Private-Key-Extract queries and q_S Sign queries. An H-query, an H_0-query, an H_1-query, a Public-Key-Gen query, a Partial-Private-Key-Extract query, and Sign query require one, one, one, one, one, and four scalar multiplications in $\mathbb{G}_1$, respectively. For an H_0-query, since $H_0(Q)$ is used repeatedly after calculating it once, it requires entirely one scalar multiplication. If we assume that a scalar multiplication in $\mathbb{G}_1$ takes T_{SM}, then the total running time is at most

$$t' \geq t + (q_H + q_{H_1} + q_{\text{PK}} + q_{\text{PPK}} + 4q_S + 1) \cdot T_{\text{SM}}.$$

■

Next, we prove unforgeability of $\mathcal{ECLS}$ (our scheme) against type II adversaries.

Theorem 2. If the ECDHP is (t', ϵ')-hard, then $\mathcal{ECLS}$ is $(t, q_H, q_{H_0}, q_{H_1}, q_{\text{PK}}, q_{\text{PPK}}, q_{\text{SK}}, q_{\text{RP}}, q_S, \varepsilon)$-existentially unforgeable against type II adversaries, for any time t and adversary's advantage ϵ such that

$$\epsilon \geq \mathsf{e}^2 \cdot (q_{\text{PK}} + 1)(q_{H_1} + 1) \cdot \epsilon'$$

$$t' \geq t + (q_H + q_{H_1} + q_{\text{PK}} + q_{\text{PPK}} + 4q_S + 1) \cdot T_{\text{SM}}.$$

Proof: Assume that an instance of the ECDHP, $(P,\ aP,\ bP)$, is given to a type II forger, $\mathcal{A}_{II}$. We will build an algorithm $\mathcal{B}$, which returns a CDH solution, abP, using $\mathcal{A}_{II}$.

[Setup]: The algorithm $\mathcal{B}$ picks a random value $s \in \mathbb{Z}_q^*$ and sets $P_{\text{pub}} = sP$ as a master public key.

The forger $\mathcal{A}_{II}$ can request to random oracles H, H_0, and H_1, and *Public-Key-Gen*, *Partial-Private-Key-Extract*, *Private-Key-Extract*, *Replace-Public-Key*, and *Sign* oracles. The algorithm $\mathcal{B}$ answers these queries via the following simulation. *[H-Queries]:* For an H-query at ID_i, $\mathcal{B}$ keeps a list of tuples (ID_i, β_i, W_i) in H-list as: $\mathcal{B}$ selects $\beta_i, \in_R \mathbb{Z}_q^*$, computes

$$W_i = \beta_i P$$

from the instance, adds a tuple (ID_i, β_i, W_i) to the H-list, and outputs $H(ID_i) = W_i$.

[H_0-Queries]: For an H_0-query at Q, $\mathcal{B}$ selects $\beta \in_R \mathbb{Z}_q^*$, calculates

$$W = \beta P$$

adds a tuple (Q, β, W) to the H_0-list, and outputs

$$H_0(Q) = W.$$

[H_1-Queries]: For an H_1-query at m_i, $\mathcal{B}$ keeps a list of tuples $(c_i, m_i, \gamma_i, W_i')$ in H_1-list as: $\mathcal{B}$ selects a random coin $c_i \in \{0, 1\}$ with $Pr[c_i = 0] = \frac{1}{q_{H_1}+1}$.

1) If $c_i = 1$, then $\mathcal{B}$ picks $\gamma_i \in_R \mathbb{Z}_q^*$, calculates $W_i' = \gamma_i P$, adds a tuple $(c_i, m_i, \gamma_i, W_i')$ to the H_1-list and outputs $H_1(m_i) = W_i'$.
2) If $c_i = 0$, then $\mathcal{B}$ picks $\gamma_i \in_R \mathbb{Z}_q^*$, calculates $W_i' = \gamma_i(bP)$ from the instance, adds a tuple $(c_i, m_i, \gamma_i, W_i')$ to the H_1-list, and outputs $H_1(m_i) = W_i'$.

[Public-Key-Gen Queries]: For this query at ID_i, $\mathcal{B}$ keeps a list of tuples (d_i, ID_i, PK_i, usk_i) in the PK-list as: $\mathcal{B}$ chooses a random coin $d_i \in \{0, 1\}$ with $Pr[d_i = 0] = \frac{1}{q_{PK}+1}$.

1) If $d_i = 0$, then $\mathcal{B}$ sets $PK_i = aP$ from the instance, adds a tuple $(d_i, ID_i, PK_i, \perp)$ to the PK-list, and outputs PK_i.
2) If $d_i = 1$, then $\mathcal{B}$ picks $x_i \in_R \mathbb{Z}_q^*$ as a private key usk_i, calculates $PK_i = x_i P$, adds a tuple (d_i, ID_i, PK_i, usk_i) to the PK-list, and outputs PK_i.

[Partial-Private-Key-Extract Queries]: For this query at ID_i, $\mathcal{B}$ keeps a list of tuples (ID_i, S_i) in the PPK-list as: $\mathcal{B}$ finds a tuple $(c_i, ID_i, \beta_i, W_i)$ from the H-list, computes

$$S_i = sW_i$$

stores (ID_i, S_i) in the PPK-list, and outputs a partial private key S_i.

[Private-Key Queries]: When $\mathcal{A}_{II}$ asks a user private key usk_i of ID_i, $\mathcal{B}$ finds the associated tuples (d_i, ID_i, PK_i, usk_i) from the PK-list.

1) If $d_i = 1$, then $\mathcal{B}$ returns usk_i.
2) If $d_i = 0$, then $\mathcal{B}$ terminates with failure.

[Replace-Public-Key Queries]: This oracle is the same as that in Theorem 1.

[Sign Queries]: When $\mathcal{A}_{II}$ requests a signature on m_i, $\mathcal{B}$ finds the associated tuples (Q, α, W) from the H_0-list and (ID_i, β_i, W_i) from the H-list, (c, m_i, γ_i, W_i') from the H_1-list, and (d_i, ID_i, PK_i, usk_i) from the PK-list,

1) If $d_i = 1$, then $\mathcal{B}$ picks $u_i, h_i \in_R \mathbb{Z}_q^*$ and computes

$$U_i = u_i P,\ V_i = sW_i + usk_i W_i' + h_i u_i W.$$

Then, $\mathcal{B}$ outputs $\sigma_i = (U_i, V_i)$.

2) If $d_i = 0$, then $\mathcal{B}$ terminates with failure.

All the outputs of the Sign queries are valid as they pass the verification equation. If $\mathcal{B}$ aborts neither a result of $\mathcal{A}_{II}$'s Sign query nor a result of $\mathcal{A}_{II}$'s Private-Key-Extract query, then $\mathcal{A}_{II}$ cannot distinguish between the real attack and the simulation. *[Forgery]:* At last, $\mathcal{A}_{II}$ returns a signature $\sigma^* = (U^*, V^*)$ on a message m^* for $\{ID_i, PK_i\}$.

1) If it is invalid, then $\mathcal{B}$ terminates with failure.
2) Otherwise, the query on ID_i and m^* already exists on the H-list, the H_1-list, and the PK-list and with tuples (ID_i, β_i, W_i), $(c_i, m_i, \gamma_i, W_i')$, and (ID_i, PK_i, usk_i), respectively. In that case, if $c_i = 1$ or $d_i = 1$, then terminates and reports failure. If $c_i = 0$ and $d_i = 0$ for some i, then the related tuples are $(d_i, ID_i, PK_i = aP, usk_i = \perp)$ and $(c_i, m^*, \beta^*, W'^* = \gamma^*(bP))$. Then, $\mathcal{B}$ computes

$$abP = \frac{V^* - sW_i - \alpha U^*}{x_i \gamma^*}.$$

At last, $\mathcal{B}$ outputs abP as a solution of the CDH instance.

We investigate the probability that $\mathcal{B}$ succeeds.

1) E_1: $\mathcal{B}$ does not abort a result of $\mathcal{A}_{II}$'s Private-Key-Extract query.
2) E_2: $\mathcal{B}$ does not abort a result of $\mathcal{A}_{II}$'s Sign query.
3) E_3: $\mathcal{A}_{II}$ outputs a valid forgery σ^* of m^* under $\{ID_i, PK_i\}$.
4) E_4: Event E_3 occurs; $c_i = 0$ and $d_i = 0$ for the tuples related to the queries on ID_i and (m_i, ID_i, PK_i) in the PK-list and the H_1-list, respectively.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

If all of these events occur, then $\mathcal{B}$ succeeds. We use the decomposition in (1) to compute the probability $Pr[E_1 \wedge E_2 \wedge E_3 \wedge E_4]$. In a similar way to Theorem 1, we get

$$Pr[E_1] \geq \left(1 - \frac{1}{q_{PK}+1}\right)^{q_{\text{PK}}}$$

$$Pr[E_2|E_1] \geq \left(1 - \frac{1}{q_{H_1}+1}\right)^{q_{H_1}}$$

$$Pr[E_3|E_1 \wedge E_2] \geq \varepsilon$$

$$Pr[E_4|E_1 \wedge E_2 \wedge E_3] \geq \frac{1}{(q_{PK}+1)}\frac{1}{(q_{H_1}+1)}.$$

Hence, from (1), $\mathcal{B}$ returns the correct answer with probability at least

$$\left(1 - \frac{1}{q_{\text{PK}}+1}\right)^{q_{\text{PK}}} \cdot \left(1 - \frac{1}{q_{H_1}+1}\right)^{q_{H_1}} \cdot \varepsilon \cdot \frac{1}{(q_{\text{PK}}+1)}\frac{1}{(q_{H_1}+1)} \geq \frac{1}{\mathbf{e}^2} \cdot \frac{\varepsilon}{(q_{\text{PK}}+1)(q_{H_1}+1)} \geq \varepsilon'. \quad \blacksquare$$

IV. Comparison and Performance Evaluation

In this section, we compare our scheme with existing CLS schemes and evaluate performance of our scheme.

A. Performance Evaluation

For performance analysis, we choose a bilinear pairing, an appropriate security level, and implementation platform.

We first choose a bilinear pairing. It is known that the bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is of the following four types [27]:

1) *Type 1:* $\mathbb{G}_1 = \mathbb{G}_2$ called symmetric pairings.
2) *Type 2:* $\mathbb{G}_1 \neq \mathbb{G}_2$, but there exists an efficient homomorphism $\phi : \mathbb{G}_2 \to \mathbb{G}_1$.
3) *Type 3:* $\mathbb{G}_1 \neq \mathbb{G}_2$ and there exist no efficient homomorphisms between $\mathbb{G}_1$ and $\mathbb{G}_2$.
4) *Type 4:* $\mathbb{G}_1 \neq \mathbb{G}_2$ and there exists an efficiently computable homomorphism between $\mathbb{G}_2$ and $\mathbb{G}_1$ with an efficient hashing way to the group element.

The results in [28] show that the Type 1 pairing defined on the finite fields of characteristic 2 or 3 is not secure. For example, the η_T pairing on supersingular curves on the fields of characteristic 2 with an embedding degree of 4 is very fast. However, it is the Type 1 pairing defined on the fields of characteristic 2, so it is still not secure. The other way for using the Type 1 pairing is to choose supersingular elliptic curves with an embedding degree of 2 over $\mathbb{F}_p$, where p is a large prime. However, the Type 1 pairings on $\mathbb{F}_p$ are very inefficient, particularly, much slower than the Type 3 pairings. Consequently, the Type 1 bilinear pairings cannot be practical solutions suitable for real applications, particularly, IIoT applications. Thus, the most efficient pairing is the asymmetric Type 3 pairing on nonsupersingular pairing-friendly curves. We use the Optimal-Ate pairing over the Barreto–Naehrig (BN) curve [29] as the bilinear pairing at a 128-bit security level, where the BN curve is a pairing friendly curve, BN-254, defined by

$$E : y^2 = x^3 + 2$$

of order N and $|N| = 254$ bits. The Optimal-Ate pairing over the BN curve is the Type 3 pairing, which is the desirable choice for an efficient and secure pairing. The Optimal-Ate pairing on BN-254 provides the same security level as RSA-3072, where RSA is the current international standard public-key signature scheme based on the integer factorization problem.

TABLE I
Efficiency Analysis of Our CLS Scheme

Operations	Specification
Pairing friendly elliptic curve	BN-254
Pairing	Optimal-Ate pairing
Security level	128-bit
Platform	Phenom II X4 940, 3.0GHz
A pairing operation (P)	0.524 ms
A scalar multiplication (SM)	0.262 ms
Sign	0.786 ms (3SM)
Sign (Precomputation)	0.262 ms (1SM)
Verify	2.096 ms (4P)
Verify (Precomputation)	1.572 ms (3P)

Our scheme requires various types of operations: a scalar multiplication on the curve, a pairing operation, a MaptoPoint operation ($H : \{0,1\}^* \times \mathbb{G}_1 \to \mathbb{G}_1$), and an addition on the curve. We consider two computational costs for SM and P since they are major operations that take the most time, where SM and P denote a scalar multiplication on the curve and a pairing operation, respectively.

We utilize the implementation results on the platform, Phenom II X4 940, 3.0 GHz in [30]. Their results are known as the fastest software implementation results:

1) *Implementation platform:* Phenom II X4 940, 3.0 GHz.
2) *Bilinear pairing:* The Optimal-Ate pairing over the BN curve as a Type 3 pairing.
3) *Security level:* 128-bit security level (it is the same security level as RSA-3072).
4) *Operation running time:* A scalar multiplication requires 0.262 ms and an Optimal Ate pairing operation requires 0.524 ms.
5) *Efficiency:* Signing and verification of our scheme require 3SM and 4P, respectively. Thus, signing and verification of our scheme take 0.786 and 2.096 ms, respectively.
6) Precomputation is possible in signing and verification.
   a) In signing, $U_i = u_i P$ and $u_i H(Q)$ can be precomputed before receiving a message as input since the values are independent of the message. Thus, signing with precomputation requires only one scalar multiplication and takes 0.262 ms.
   b) In verification, the server can precompute the value $e(H(ID_i), P_{\text{pubb}})$ for frequently used ID information, which is independent of the messages being signed and signature value. Thus, verification with precomputation requires three pairing operations and takes 1.572 ms.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF THE CLS SCHEMES

CLS Scheme	Du et al. [31]	Thumbur et al. [11]	Deng et al. [32]	Deng et al. [33]	Thumbur et al. [34]	$\mathcal{ECLS}$
Sign	1 SM	1 SM	2 SM	3 SM	1 SM	3 SM
Verify	4 SM	3 SM	3P+SM	3P+3SM	4 SM	4 P
PK Size	$2\|\mathbb{G}_1\|$	$2\|\mathbb{G}_1\|$	$\|\mathbb{G}_1\|$	$2\|\mathbb{G}_1\|$	$2\|\mathbb{G}_1\|+q$	$\|\mathbb{G}_1\|$
Sig. Size	$\|\mathbb{G}_1\|+q$	$\|\mathbb{G}_1\|+q$	$4\|\mathbb{G}_1\|$	$2\|\mathbb{G}_1\|$	$2\|\mathbb{G}_1\|+q$	$2\|\mathbb{G}_1\|$
Total communication cost	$3\|\mathbb{G}\|+q$	$3\|\mathbb{G}\|+q$	$5\|\mathbb{G}_1\|$	$4\|\mathbb{G}_1\|$	$4\|\mathbb{G}_1\|+q$	$3\|\mathbb{G}_1\|$
Cryptanalysis	$\vee$	Type I attack [21]	Type I attack	Type II attack [36]	Type I attack	$\vee$

We summarize efficiency analysis of our scheme in Table I. As seen in Table I, a pairing operation is two times slower than a scalar multiplication. Thus, the pairing-free CLS schemes are faster than the pairing-based CLS schemes. However, [12, Table III] shows that the existing pairing-free CLS schemes are insecure against the designated attacks.

B. Comparison

Owing to the suitability of CLS schemes for IoT environments, similar CLS schemes have been proposed for a long time and, despite their security proofs, they have been attacked and modified to prevent the attacks. Now, we compare our scheme with the existing CLS schemes [11], [31], [32], [33], [34] in terms of computational cost and total communication cost in Table II, where PK and Sig. are abbreviations for public key and signature, respectively. The "Cryptanalysis" row in Table II represents whether the CLS scheme is secure against the designated attack and $\vee$ means that there is no known attacks.

Our scheme has the lowest total communication cost among the schemes, but its signing and verification are inefficient. It is because our scheme is designed to be proved without requiring the forking lemma using three hash functions as random oracles, so it provides stronger security guarantee than the CLS schemes using the forking lemma. It is a strength in terms of security, but the use of an additional hash function requires additional computations for the pairing operation. In Table II, Du et al.'s scheme is more efficient than our scheme; their scheme still uses the forking lemma and cannot provide aggregation property, which allows us to aggregate n different signatures on n different messages of n different nodes into a single signature, which reduces communication overhead and computational cost. In our scheme, a given n signatures $\{sigma_i = (U_i, V_i)\}_{i=1}^n$ on $\{m_i\}_{i=1}^n$ under $\{ID_i, PK_i\}_{i=1}^n$, they can be aggregated as the following single signature:

$$\sigma_{\text{agg}} = \left(U_1, \cdots, V = \sum_{i=1}^{n} V_i\right).$$

Then, communication cost can be significantly reduced from $2n$ elements of the underlying group as n individual signatures into $(n+1)$ elements. In Du et al.'s scheme, they used k^{-1} in the individual signature generation, where k is a random number, which makes the aggregation impossible.

V. CONCLUSION

Recently, Wang et al. [12] pointed out several security weaknesses on most of the existing CLS schemes and then constructed a novel efficient CLS scheme based on the blockchain technique and smart contract. In this article, we showed that Wang et al.'s scheme was broken by the key recovery attacks: anyone could recover the entire private keys of any users from only two signatures. We then proposed a new CLS scheme to prevent our attacks and proved the unforgeability of our scheme against type I and type II adversaries under the hardness assumption of the ECDHP without using the forking lemma.

REFERENCES

[1] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. IT-22, no. 6, pp. 644–654, Nov. 1976.

[2] A. Shamir, "Identity-based cryptosystems and signature schemes," in *Proc. Workshop Theory Appl. Cryptographic Techn.*, 1984, pp. 47–53.

[3] S. S. Al-Riyami and K. G. Patersony, "Certificateless public key cryptography," in *Advances in Cryptography*. Berlin, Germany: Springer-Verlag, 2003, pp. 452–473.

[4] L. Wang, K. Chen, Y. Long, and H. Wang, "An efficient pairing-free certificateless signature scheme for resource-limited systems," *Sci. China Inf. Sci.*, vol. 60, no. 11, 2017.

[5] K. H. Yeh, "A secure transaction scheme with certificateless cryptographic primitives for IoT-based mobile payments," *IEEE Syst. J.*, vol. 12, no. 2, pp. 2027–2038, Jun. 2018.

[6] X. Jia, D. He, Q. Liu, and K.-K. R. Choo, "An efficient provably-secure certificateless signature scheme for Internet-of-Things deployment," *Ad Hoc Netw.*, vol. 71, pp. 78–87, 2018.

[7] A. Karati, S. H. Islam, and M. Karuppiah, "Provably secure and lightweight certificateless signature scheme for IIoT environments," *IEEE Trans. Ind. Inform.*, vol. 14, no. 8, pp. 3701–3711, Aug. 2018.

[8] M. E. S. Saeed, Q.-Y. Liu, G. Tian, B. Gao, and F. Li, "Remote authentication schemes for wireless body area networks based on the Internet of Things," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4926–4944, Dec. 2018.

[9] Y.-F. Tseng, C.-I. Fan, and C.-W. Chen, "Top-level secure certificateless signature scheme in the standard model," *IEEE Syst. J.*, vol. 13, no. 3, pp. 2763–2774, Sep. 2019.

[10] Y. Zhang, R. H. Deng, D. Zheng, J. Li, P. Wu, and J. Cao, "Efficient and robust certificateless signature for data crowdsensing in cloud-assisted industrial IoT," *IEEE Trans. Ind. Inform.*, vol. 15, no. 9, pp. 5099–5108, Sep. 2019.

[11] G. Thumbur, G. S. Rao, P. V. Reddy, N. Gayathri, and D. R. K. Reddy, "Efficient pairing-free certificateless signature scheme for secure communication in resource-constrained devices," *IEEE Commun. Lett.*, vol. 24, no. 8, pp. 1641–1645, Aug. 2020.

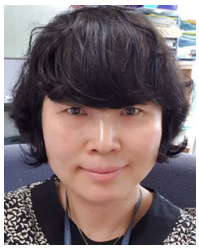
[12] W. Wang, H. Xu, M. Alazab, T. R. Gadekallu, Z. Han, and C. Su, "Blockchain-based reliable and efficient certificateless signature for IIoT devices," *IEEE Trans. Ind. Inform.*, vol. 18, no. 10, pp. 7059–7067, Oct. 2022.

[13] D. H. Yum and P. J. Lee, "Generic construction of certificateless signature," in *Proc. Australas. Conf. Inf. Secur. Privacy*, 2004, pp. 200–211.

[14] M. H. Au, J. Chen, J. K. Liu, Y. Mu, D. S. Wong, and G. Yang, "Malicious KGC attacks in certificateless cryptography," in *Proc. ACM Symp. Inf. Comput. Commun. Secur.*, 2007, pp. 302–311.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.

[15] P. Gong and P. Li, "Further improvement of a certificateless signature scheme without pairing," *Int. J. Commun. Syst.*, vol. 27, no. 10, pp. 2083–2091, 2014.
[16] K. A. Shim, "Universal forgery attacks on remote authentication schemes for wireless body area networks based on Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 5973–5984, Oct. 2019.
[17] J. Liu, L. Wang, and Y. Yu, "Improved security of a pairing-free certificateless aggregate signature in healthcare wireless medical sensor networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5256–5266, Jun. 2020.
[18] K. A. Shim, "Forgery attacks on two provably secure certificateless signature schemes," *Inf. Sci.*, vol. 521, pp. 81–87, 2020.
[19] K. A. Shim, "Cryptanalysis of two signature schemes for IoT-based mobile payments and healthcare wireless medical sensor networks," *IEEE Access*, vol. 8, pp. 167203–167208, 2020.
[20] Y. Liao, Y. Liu, Y. Liang, Y. Wu, and X. Nie, "Revisit of certificateless signature scheme used to remote authentication schemes for wireless body area networks," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2160–2168, Mar. 2020.
[21] Z. Xu, M. Luo, M. K. Khan, K. K. R. Choo, and D. He, "Analysis and improvement of a certificateless signature scheme for resource-constrained scenarios," *IEEE Commun. Lett.*, vol. 25, no. 4, pp. 1074–1078, Apr. 2021.
[22] L. Zhang, Z. Zhang, W. Wang, Z. Jin, Y. Su, and H. Chen, "Research on a covert communication model realized by using smart contracts in blockchain environment," *IEEE Syst. J.*, vol. 16, no. 2, pp. 2822–2833, Jun. 2022, doi: 10.1109/JSYST.2021.3057333.
[23] L. Zhang, Y. Zou, W. Wang, Z. Jin, Y. Su, and H. Chen, "Resource allocation and trust computing for blockchain-enabled edge computing system," *Comput. Secur.*, vol. 105, 2021.
[24] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, no. 3, pp. 361–396, 2000.
[25] D. Abbasinezhad-Mood, S. M. Mazinani, M. Nikooghadam, and A. O. Sharif, "Efficient provably-secure dynamic ID-based authenticated key agreement scheme with enhanced security provision," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1227–1238, Mar. 2022, doi: 10.1109/TDSC.2020.3024654.
[26] K. A. Shim, "A new certificateless signature scheme provably secure in the standard model," *IEEE Syst. J.*, vol. 13, no. 2, pp. 1421–1430, 2019.
[27] S. D. Galbraith, K. G. Paterson, and N. P. Smart, "Pairings for cryptographers," *Discrete Appl. Math.*, vol. 156, no. 16, pp. 3113–3121, 2008.
[28] S. D. Galbraith and P. Gaudry, "Recent progress on the elliptic curve discrete logarithm problem," *Designs, Codes Cryptogr.*, vol. 78, no. 1, pp. 51–72, 2015.
[29] P. S. L. M. Barreto and M. Naehrig, "Pairing-friendly elliptic curves of prime order," in *Proc. Int. Workshop Sel. Areas Cryptogr.*, 2005, pp. 319–331.
[30] D. F. Aranha, K. Karabina, P. Longa, C. H. Gebotys, and J. Lopez, "Faster explicit formulas for computing pairings over ordinary curves," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2011, pp. 48–68.
[31] H. Du, Q. Wen, S. Zhang, and M. Gao, "A new provably secure certificateless signature scheme for Internet of Things," *Ad Hoc Netw.*, vol. 100, 2020, Art. no. 102074.
[32] L. Deng, Y. Yang, and Y. Chen, "Certificateless short aggregate signature scheme for mobile devices," *IEEE Access*, vol. 7, pp. 87162–87168, 2019.
[33] L. Deng, B. Ning, and Y. Jiang, "A lightweight certificateless aggregation signature scheme with provably security in the standard model," *IEEE Syst. J.*, vol. 14, no. 3, pp. 4242–42518, Sep. 2020.
[34] G. Thumbur, P. V. Reddy, G. S. Rao, N. B. Gayathri, D. V. R. K. Reddy, and M. Padmavathamm, "Efficient and secure certificateless aggregate signature based authentication scheme for vehicular adhoc networks," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1908–1920, Feb. 2021.
[35] K. A. Shim, "Design principles of secure certificateless signature and aggregate signature schemes for IoT environments," *IEEE Access*, vol. 10, pp. 124848–124856, 2022.



Kyung-Ah Shim (Member, IEEE) received the Ph.D. degree in mathematics from Ewha Womans University, Seoul, South Korea, in 1999.

She is currently a Senior Researcher with the National Institute for Mathematical Sciences, Daejeon, South Korea. Her research interests include public-key cryptography, post-quantum cryptography, cryptographic protocols, and information security.

Authorized licensed use limited to: Chaitanya Bharathi Institute of Tech - HYDERABAD. Downloaded on April 29,2026 at 11:02:33 UTC from IEEE Xplore. Restrictions apply.